

КАК УСТАНОВИТЬ РОБОТА НА СВОЕМ КОМПЬЮТЕРЕ И НАЧАТЬ РАБОТУ

1. Установите **Julia**, запустив установщик с [официального сайта](#)
2. Установите интегрированную среду разработки **VS Code** с [официального сайта](#)
3. Откройте **VS Code** и установите расширение для поддержки языка **Julia**
4. Зпустите **Julia** из командной строки **VS Code** - в результате появится командная строка интерактивной среды (**REPL Julia**)
5. Переведите командную строку **Julia** в режим пакетного менеджера нажатием клавиши **]**
6. Из командной строки пакетного менеджера запустите установку пакета **GLMakie**:

```
pkg> add GLMakie
```

7. После завершения установки из той же командной строки пакетного менеджера установите пакет **HorizonSideRobots**:

```
pkg> add https://github.com/Vibof/HorizonSideRobots.jl
```

8. После завершения установки из режима пакетного менеджера вернитесь в обычный режим командной строки **REPL Julia** нажатием клавиши **Backspace**

Если все прошло нормально, то установленные пакеты могут быть импортированы, например, непосредственно в интерактивную среду **REPL Julia**:

```
julia> using GLMakie  
julia> using HorizonSideRobots
```

(последовательность импортов значения не имеет).

Важно чтобы среда **VS Code** была настроена таким образом, что при построении графика он выводился бы в отдельном окне, а не в имеющейся специальной встроенной панели для графиков среды **VS Code**.

После этого Робота можно будет запустить непосредственно из командной строки интерактивной среды:

```
julia> r = Robot(animate = true)
```

В результате выполнения этой команды откроется окно с Роботом на клетчатом поле, ограниченном прямоугольной рамкой. Переменной `r` при этом будет присвоено значение ссылки на объект типа `Robot`.

Важно отметить, что в режиме визуализации пакет рассчитан на работу только с одним отображаемым Роботом одновременно.

Значение именованного аргумента конструктора `animate = true` означает, что Робот создается в режиме анимации, т.е. в режиме позволяющем визуально наблюдать его перемещения по полю. Соответственно, в случае `animate = false` наблюдать Робота и его перемещения будет невозможно.

Запускать Робота без визуализации следует, например, когда требуется исключить временные задержки, связанные с визуализацией, т.к. при достаточно больших размерах поля эти задержки могут оказаться весьма значительными.

С другой стороны, режим с визуализацией удобен при отладке, т.к. в этом случае имеется возможность визуально наблюдать пошаговое выполнение программы.

Размеры поля можно задать произвольными, например, поле 20 на 25 клеток можно создать командой:

```
julia> r = Robot(20, 25, animate = true)
```

С помощью мыши на поле можно создать произвольную начальную обстановку.

Так, Робота можно переместить в любую видимую клетку поля. Для этого надо сначала кликнуть по телу робота - его цвет изменится на более светлый, затем кликнуть по целевой клетке - робот автоматически переместится в требуемую клетку.

Кликав мышью по центру любой клетки, можно установить/убрать маркер, кликая вблизи границы между соседними клетками, можно установить/убрать перегородку между этими клетками.

Если кликнуть в любом месте вблизи внешней рамки поля, то можно убрать/восстановить внешнюю границу поля. При снятии внешней ограничивающей рамки поле становится бесконечным во все стороны. Это означает, что Робот сможет перемещаться за пределы видимой части поля, а потом возвращаться назад. При этом, если какая-либо внутренняя перегородка будет доходить вплотную до границы видимой части поля, то считается, что эта граница прямолинейно продолжается до бесконечности в соответствующую сторону.

Сразу после первого клика мышью для изменения обстановки на поле автоматически создается файл `untitled.sit`, в котором сохраняется создаваемая обстановка. Впоследствии этот файл можно переименовать и сохранить, например, для тестирования программы.

Для получения справочной информации о командном интерфейсе Робота можно непосредственно в интерактивной среде `Julia`, воспользовавшись встроенной справочной системой. Для этого

сначала надо перейти в соответствующий режим командной строки, выполнив команду:

```
julia> ?
```

в результате чего приглашение командной строки вид:

```
help?>
```

и далее:

```
help?> Robot
search: Robot

r = Robot(;animate=false)

r = Robot(num_rows::Integer, num_colons::Integer; animate=false)

r = Robot(file_name::AbstractString; animate=false)

- Конструктор объектов типа Robot (исполнитель "Робот на клетчатом поле со
сторонами горизонта типа HorizonSide"):

. - в первом случае - пустое ограниченное поле будет иметь размер 11x12 и
Робот будет - в левом нижнем углу

. - во втором случае - поле будет иметь указанные размеры

. - в третьем случае - обстановка загружается из указанного файла (обычно - с
расширением sit)

- Если animate=true, то при этом открывается окно с соответствующей начальной
обстановкой (которую можно будет редактировать, и результат редактирования
будет сохраняться в файле "untitled.sit"). Иначе просто создается объект,
содержащий данные, определяющие эту обстановку (визуализации обстановки в этом
случае нет).

Командный интерфейс исполнителя (объекта) типа Robot: move!, isborder,
putmarker, ismarker, temperature, show_sit, show_sit! (см. help)

Для подготовительной работы с sit-файлами (в них сохраняется данные с
информацией о некоторой обстановке на
поле с Роботом) имеются специальные функции: show_sit!, edit_sit, create_sit
(см. help)
```

Для возможности получения этой информации предварительно пакет `HorizonSideRobots` должен быть импортирован.

Для получения информации о назначении и способах использования всех импортированных из пакета функций можно также воспользоваться встроенной системой помощи, например:

```
help?> move!
search: move! merge!

move!(r::Robot, side::HorizonSide)::Nothing

- Перемещает Робота в соседнюю клетку в заданном направлении (если только на
пути нет перегородки, в противном
случае - прерывание)
```

(важно отметить, что **!** в конце имени некоторых функций - это часть имени каждой из таких функций).

Для удобства рекомендуется создать файл `start.jl` со скриптом:

```
# start.jl

using GLMakie
using HorizonSideRobots

robot = Robot(animate = true)
```

Если этот файл будет находиться в папке проекта, и эта папка будет сделана текущей (путем выбора ее в встроенном файловом менеджере **VS Code**), этот скрипт может быть выполнен, например, из командной строки с помощью стандартной библиотечной функции `include`:

```
julia> include("start.jl")
```

Это равнозначно тому, как если бы команды содержащиеся в данном скрипте вводились бы непосредственно в командной строке.

Если `test_1.sit` - это файл с некоторой начальной обстановкой на поле, то скрипт `start.jl` мог бы выглядеть так:

```
# start.jl

using GLMakie
using HorizonSideRobots

robot = Robot("test.sit", animate = true)
```

И тогда, поскольку в этом случае уже не потребуется вручную редактировать начальную обстановку на поле, то функция `include("start.jl")` может быть помещена сразу в файл вместе с исполняемым программным кодом. При этом сам файл с кодом может быть вызван также запущен на исполнение из командной строки с помощью той же функции `include("test_1.jl")`, где `test_1.jl` - имя файла с программным кодом, находящемся в текущей папке (рядом с файлами `start.jl` и `test_1.sit`).

Например, содержимое файла `teast_1.jl` может быть таким:

```
# test_1.jl

include("start.jl")
include("lib_rob.jl")
prog(robot)
```

где `lib_rob.jl` - файл, содержащий определение некоторой пользовательской функции `prog`, в которую ссылка на созданный в `start.jl` объект типа `Robot`, содержащаяся в переменной `robot`, передается в функцию `prog` как аргумент.

Предполагается, что файл `lib_rob.jl` также находится в текущей папке рядом с остальными файлами (либо должен быть указан полный путь к этому файлу).

Таким образом, запуск скрипта `test_1.jl` может быть осуществлен, например, так:

```
julia> include("test_1.jl")
```